Qwen

Architecting Deterministic Combat: A Technical Blueprint for the EngAIn Combat & Magic Specification v0.1

Foundational Determinism and Simulation Architecture

The successful implementation of the EngAIn Combat & Magic Specification v0.1 hinges upon establishing a foundational architecture that guarantees strict determinism across all simulation components [27]. Determinism, defined as the principle where a simulation produces the exact same game state on all machines when given the same initial conditions and input sequence, is not merely a feature but the bedrock upon which all other mechanics—Lightning propagation, elemental reactions, and AoE targeting—must be built [8] [24]. The requirement for "strict reproducibility," particularly for deterministic replay, dictates that common sources of non-determinism inherent in standard game engines must be systematically eliminated or replaced with controlled, portable alternatives [5] [21]. This involves a holistic approach addressing time management, numerical precision, randomness, and data structure integrity.

A primary pillar of this architecture is the enforcement of a fixed, deterministic simulation timestep. Many game engines utilize a variable `DeltaTime` derived from the rendering loop, which fluctuates with graphical load and leads to inconsistent physics calculations and non-reproducible behavior [5]. To achieve determinism, the EngAIn platform must operate on a single, constant `FixedDeltaTime` for all authoritative game logic updates [45]. This ensures that every simulation step is executed identically regardless of the host machine's performance, preventing the "butterfly effect" where small timing differences propagate into large state divergences [3]. This principle is mirrored in frameworks like Unreal Engine's substepping and custom solutions like Balaur, which enforce a stable tick rate to create a reliable simulation environment [5] [45].

The second critical component is the replacement of standard floating-point arithmetic (`float`, `double`) with a deterministic fixed-point math library. Floating-point operations are notoriously unreliable for networked simulations because different hardware architectures (e.g., Intel vs. AMD CPUs) and compilers can produce slightly different results due to variations in rounding modes, intermediate precision, and

implementations of transcendental functions like `sin` and `cos` [4] [7]. Even IEEE 754 compliance does not guarantee bit-for-bit identical results for all operations, especially transcendental ones [4]. The solution is to mandate the use of fixed-point arithmetic for all simulation-critical calculations, including collision detection, pathfinding, and damage computations. Libraries such as FixedMathSharp or Metaplay's SDK provide engine-agnostic, portable numeric types like `F64` (a signed 32.32 format) or `Fixed64` that guarantee identical outputs for the same inputs across all platforms [22] [23]. These libraries typically represent numbers as large integers scaled by a power of two, ensuring that all operations are performed using pure integer math, which is universally consistent [50] [51]. Conversions back to floating-point for rendering or UI purposes should only occur at the very boundary of the system to avoid introducing non-determinism into the core logic [22].

Thirdly, any element of chance within the simulation must be fully deterministic. Relying on platform-specific random number generators or system entropy is unacceptable [27]. Instead, the platform must employ a seeded pseudo-random number generator (PRNG). The initial seed for the PRNG becomes part of the initial state of the simulation, ensuring that two instances of the simulation starting from the same seed and receiving the same input sequence will generate identical streams of "random" numbers [5]. For complex systems involving LLMs or external APIs, the framework must intercept and log all interactions, replacing live calls with deterministic stubs that return the exact recorded responses during a replay [35]. In the context of EngAIn, this means that for every random decision point—for example, choosing a branch in a Lightning spell's propagation—the current state of the PRNG must be captured along with the action taken [45]. During a replay, this stored state is used to reproduce the exact outcome, ensuring that different rendering clients never independently determine combat randomness [45].

Finally, the choice of data structures is paramount for maintaining determinism. Sequential processing of collections whose order affects game state can lead to divergence if unordered structures like hash maps are used, as their iteration order is not guaranteed to be consistent across runs [8]. The platform must therefore mandate the use of ordered or sorted data structures (e.g., balanced binary trees or sorted lists) for any collection whose elements are processed in a sequence that could alter the simulation's outcome. This ensures that logic paths remain predictable and reproducible. The combination of a fixed timestep, fixed-point arithmetic, a seeded PRNG, and ordered data structures creates a robust foundation capable of supporting the complex, interdependent mechanics specified in the document while guaranteeing the strict reproducibility required for calibration and replay.

To formalize and validate this commitment to determinism, a strict replay contract must be established. Drawing inspiration from best practices in projects like Conquestoria, this contract should be enforced through a series of regression tests that verify the simulation's deterministic properties [27]. Four key test contracts are essential:

1. **Same Seed + Same Commands:** This test verifies that two fresh simulations initiated with the same seed and provided with an identical sequence of player commands must reach bitwise equivalent states. This is the most fundamental test of the system's core logic.
2. **Save/Load Continuity:** This test ensures that running a simulation for N turns, saving the state, loading it, and continuing for another M turns produces the same final state as an uninterrupted run of N+M turns. This validates that the serialization and deserialization processes are lossless and deterministic.
3. **Deterministic AI:** Given identical seeds, game states, and simulated perceptions, an AI-controlled entity must make the exact same choices and result in the same state, even after save/load boundaries. This isolates the AI logic from the main game state and ensures its decisions are also reproducible.
4. **Domain-stream Stability:** When a new stochastic subsystem is introduced, it must not perturb unrelated domains. For instance, consuming a random value from the Lightning propagation stream should leave the random state of the elemental reaction kernel untouched, and vice versa. This prevents unintended interactions between independent systems.

These tests form the basis of an automated verification suite that must be integrated into the CI/CD pipeline to prevent regressions that could introduce non-determinism [15]. By enforcing these architectural constraints and validation contracts, the EngAIn platform can reliably execute the specified combat and magic mechanics with the precision and predictability demanded by competitive balancing and deterministic replay.

Core Data Contracts for Authorable Gameplay

The EngAIn architecture is predicated on a clear separation between reusable simulation kernels and externally authored content definitions [13]. This modularity allows authors to define a vast array of unique abilities and world states without requiring changes to the underlying engine code. This section defines the concrete, engine-independent data contracts that serve as the blueprint for this authorable gameplay. These contracts are machine-readable agreements that specify the structure, semantics, and constraints of the

data exchanged between producers (authors, level designers) and consumers (the simulation kernels) [15]. They distinguish between parameters governed by universal algorithms and rules that are subject to authorial discretion.

A foundational contract is the `PlatformGeometryContract`. This contract defines the spatial constraints and physical properties of a combat arena, forming the static backdrop against which all dynamic actions unfold. Its fields would include:

- `grid_resolution: Vector3i`: Defines the size of the discrete grid cells that compose the platform, crucial for spatial hashing and line-of-sight calculations.
- `platform_bounds: AABB`: An axis-aligned bounding box that delineates the total physical extent of the platform.
- `obstacle_layers: List[LayerId]`: A list of entity layer identifiers that are considered solid obstacles, preventing traversal by characters or projectiles unless they possess specific abilities.

This contract would be consumed by a spatial query kernel, which uses its information to accelerate collision detection and pathfinding. For efficiency, a sparse spatial hash data structure is recommended, mapping objects to grid cells to minimize the number of pairwise checks required [36].

The central contract for magic is the `SpellDefinitionContract`. It encapsulates all properties of a spell, separating generic algorithmic parameters from author-defined rules. Inspired by systems like SimulationCraft's override system, this contract enables deep customization while preserving a consistent execution framework [14]. Its structure would be hierarchical:

- `base_data`: Contains general, unchanging properties like `name`, `cast_time`, `cooldown`, and resource cost (`cost`, `pct_cost`).
- `effect_data`: Specifies the actionable properties of the spell's impact, such as damage scaling coefficients (`sp_coefficient`, `ap_coefficient`), healing amounts, aura application values, and duration [14].
- `propagation_rules`: Defines the behavioral logic for how the spell's effect travels through space and interacts with targets. This is the heart of authorial control. Key fields include:
- `contact_resolution_strategy: enum { IGNORE_REPEATED, AMPLIFY_REPEATED, APPLY_DAMAGE_REPEATED }`: This directly addresses the user's requirement for a configurable mapping strategy. It allows an author to decide whether repeated pad traversal on the same enemy should be ignored, cause amplified damage, or

apply damage each time. This preserves the distinction between pad traversal and actual HP damage [13].
- `max_chain_length: int`: The maximum number of sequential hits a spell like Lightning can achieve before terminating.
- `branching_factor: float`: A mechanism or probability for generating branches at junctions.
- `reaction_cooldown: Duration`: A delay before a target hit by this spell can trigger another elemental reaction with the same element, preventing infinite loops.

This contract-driven design ensures that the core propagation kernel remains generic and reusable; it applies the effects defined in `effect_data` according to the rules specified in `propagation_rules`.

For persistent elemental states, the `AuraContract` provides a standardized way to represent them. This contract models the "Elemental Gauge Theory" described in related materials, where an element's strength is measured in quantifiable units [42] [43]. Its fields are:

- `element_type: ElementType`: The type of element applied (e.g., Pyro, Cryo, Hydro).
- `gauge_units: F64`: The strength of the aura, representing both its immediate effect and its remaining duration. One Gauge Unit is equivalent to 25 units of Element Durability [42].
- `decay_rate_seconds_per_gu: F64`: The rate at which the aura's gauge decays over time, determining its total lifespan.
- `creation_timestamp: SimulationTimestamp`: The point in the simulation when the aura was first applied, used for calculating its remaining duration.

The kernel responsible for elemental reactions reads and modifies this contract. For example, applying a new aura might involve an "Aura Tax" that reduces the incoming gauge by 20% before storing it [42]. This clear contract makes the state of auras explicit, observable, and manageable by the simulation logic.

Finally, terrain effects like Mudstorm are defined through a `TerrainEffectContract`. These effects are not tied to individual entities but are permanent alterations to the platform itself, making them a form of durable, authored content [13]. The contract might look like:

- `effect_type: TerrainEffectType`: Identifies the type of effect (e.g., Mudstorm).

- `pattern_definition: GridPattern`: A set of coordinates on the platform's grid where the effect is active.
- `author_metadata`: Information about who created the effect.

A `TerrainEffectManager` kernel would read these contracts to populate a persistent state, which other kernels (like the propagator) can query to modify their behavior. For instance, the propagator would check if a potential landing pad is within a Mudstorm pattern and, if so, alter the spell's branching probabilities accordingly. This approach keeps the platform's geometry and authored effects decoupled from the dynamic entity state, allowing levels to be designed independently of character abilities. Together, these contracts provide a comprehensive, extensible schema for defining the entire scope of the combat and magic system in a way that is both powerful for authors and robust for the simulation engine.

Directional Lightning Propagation and Combo Mechanics

The implementation of directional Lightning propagation and combo qualification requires the development of specialized, yet reusable, execution algorithms that consume the data contracts defined previously. These algorithms must be engineered to be deterministic, respecting the EngAIn platform's core principles while providing the flexibility needed for diverse spell behaviors and combat scenarios. The analysis distinguishes between the universal logic governing these mechanics and the author-configurable parameters that allow for unique expression.

The `DirectionalLightningPropagator` algorithm is responsible for simulating the path of a Lightning spell from its source. Its operation is a breadth-first search through the platform's contact pads, constrained by the spell's propagation rules. The algorithm begins by placing the source pad onto a priority queue, which prioritizes paths based on their length and the number of remaining branches. As the algorithm dequeues a path, it identifies adjacent pads that are in range and not obstructed. For each valid target pad, it consults the `contact_resolution_strategy` parameter from the `SpellDefinitionContract` to determine if a hit should be registered. If the strategy is `IGNORE_REPEATED`, the algorithm checks if the target entity has already been hit in the current chain and skips it if so. If the strategy is `AMPLIFY_REPEATED` or `APPLY_DAMAGE_REPEATED`, the hit is registered regardless of prior contact. Once a hit is confirmed, the target entity is added to a set of visited entities, the hit is recorded in

the results, and a new path extending to the target pad is enqueued for further propagation, decrementing the remaining branch count [13]. This process continues until the queue is empty or the maximum number of hits is reached.

A critical aspect of this algorithm is its handling of randomness at branching points. At any junction where multiple valid paths exist, the propagator must make a choice. To ensure determinism, this choice cannot be made randomly at runtime. Instead, the system must record the decision. During the initial authoritative simulation, the PRNG state is sampled at the moment of the decision, and the chosen path is logged [45]. When a replay is initiated, the algorithm retrieves this pre-recorded choice from the replay file instead of re-randomizing, guaranteeing an identical propagation path [27]. This strict ownership of random decisions by the authoritative server is a cornerstone of the platform's reproducibility contract [1]. The algorithm must also incorporate termination conditions, such as the `max_chain_length` rule, which causes propagation to cease once a certain number of hits have occurred, effectively modeling the "triple-chain termination" mechanic mentioned in the research goal [13].

Pseudocode	Description
`queue = PriorityQueue()`	Initialize a queue to manage paths under exploration, prioritizing shorter paths.
`visited_entities = Set()`	Track entities already hit in the current chain to enforce `contact_resolution_strategy`.
`path_queue.push((source_pad, max_branches, [source_pad]))`	Seed the queue with the starting position of the spell.
`spell_config = getSpellDefinition(spell_id)`	Fetch the author-defined rules for this specific spell.
`if branches_left == 0: continue`	Skip processing if no more branches can be created.
`adjacent_pads = findAdjacentPadContacts(current_pad)`	Find all pads immediately adjacent to the current one.
`valid_targets = filter_obstructed(adjacent_pads)`	Remove any targets that are blocked from the path.
`target_entity = getEntityFromPad(target_pad)`	Retrieve the entity occupying the target pad.
`should_hit = resolveContactStrategy(spell_config, target_entity, visited_entities)`	Apply the author's rule for handling repeated contacts.
`results.add((target_pad, path))`	Record the successful hit and its path.
`visited_entities.add(target_entity)`	Update the set of visited entities.
`new_path = path + [target_pad]`	Extend the path to include the newly hit pad.
`queue.push((target_pad, new_branches, new_path))`	Enqueue the new path for further propagation.

In parallel, the `FrameAdvantageCalculator` provides the technical basis for combo qualification. This algorithm determines if an attacking move can be followed by a

subsequent move by comparing their respective timings. The calculation is based on the frame data of the moves involved. The formula is `Frame Advantage = (Attacker's Active Frames + Defender's Hitstun Frames) - Attacker's Recovery Frames` [29]. An attack that leaves the defender in hitstun longer than it takes the attacker to recover is considered "plus" on hit. A follow-up attack can be successfully chained if its startup frames are less than or equal to the frame advantage gained from the previous hit [29]. This principle governs the flow of combos in fighting games and provides a deterministic, calculable metric for evaluating attack sequences.

Pseudocode	Description
`attacker_move_frame_data = getMoveFrameData(move_id)`	Retrieve the timing data for the attacking move.
`defender_move_frame_data = getMoveFrameData(blocked_move_id)`	Retrieve the timing data for the move that was blocked.
`hitstun_frames = attacker_move_frame_data.active_frames + defender_move_frame_data.hitstun_frames`	Calculate the total duration of the defender's incapacitated state.
`recovery_frames = attacker_move_frame_data.recovery_frames`	Determine how many frames the attacker needs to recover.
`frame_advantage = hitstun_frames - recovery_frames`	Compute the net frame advantage or disadvantage.
`return frame_advantage`	Return the calculated advantage value.

These two algorithms, while serving different purposes, share a common design philosophy: they are generic frameworks that consume specific data. The `DirectionalLightningPropagator` consumes a `SpellDefinitionContract` to dictate its branching and termination logic, while the `FrameAdvantageCalculator` consumes a `MoveFrameData` contract to perform its timing calculations. This modular approach ensures that the core simulation logic remains stable and reusable, while the rich variety of gameplay expressions comes from the externally authored definitions, perfectly aligning with the EngAIn architecture's goals.

Elemental Reaction Modeling and Persistent Effects

The EngAIn specification requires a sophisticated system for modeling elemental reactions and persistent terrain effects, both of which rely on a shared understanding of the game world's state. The implementation of these features is driven by dedicated kernels that interpret authorable data contracts to modify entity states and influence future actions. This approach maintains the platform's separation of concerns, where the

simulation logic is distinct from the specific rules of engagement defined by the content creators.

The `ElementalReactionKernel` is the central component for handling all interactions between elemental attacks and the auras they create on targets. This kernel operates on the `AuraContract`, which represents a persistent elemental state attached to an entity [42]. When a spell with a reactive element strikes a target, the kernel first queries the target for an existing aura of the triggering element's type. If no aura is present, no reaction occurs. If an aura exists, the kernel calculates the outcome based on the reaction type and a predefined coefficient [42]. The core logic follows a simple formula: `New_Gauge = Aura_Gauge - (Trigger_Gauge * Coefficient)`. If the resulting gauge is greater than zero, the aura persists with its new, reduced strength. If the gauge drops to zero or below, the aura is completely consumed and removed from the target [42].

This general model accommodates a wide range of reactions through different coefficients and special-case logic. For instance, Transformative Reactions like Overloaded and Superconduct have a coefficient of 1.0, meaning they consume a 1:1 ratio of the trigger's gauge from the aura [43]. Amplifying Reactions like Melt and Vaporize have a higher coefficient (e.g., 2.0), causing them to consume more gauge than they contribute, resulting in a larger damage output [43]. Special reactions require more complex handling. The `Electro-Charged` reaction, for example, is tick-based and persists as long as both Hydro and Electro gauges are present. Each tick consumes 0.4U from both elements simultaneously until one is depleted [42]. The `Frozen` reaction is unique in that it consumes both Cryo and Hydro but then replaces them with a new, temporary "Freeze Aura" that can be shattered by physical attacks [42]. The kernel's implementation must contain explicit checks for these special cases before applying the default consumption formula.

Reaction Type	Trigger Element	Target Aura	Coefficient	Outcome Description
Melt / Vaporize	Pyro / Hydro	Pyro / Hydro	2.0	Consumes twice the trigger's gauge, amplifying damage.
Crystallize / Swirl	Geo / Anemo	Any	0.5	Consumes half the trigger's gauge, often creating a new effect.
Overloaded / Superconduct	Pyro / Cryo	Pyro / Cryo	1.0	Consumes a 1:1 ratio of the trigger's gauge from the aura.
Electro-Charged	Electro / Hydro	Electro / Hydro	0.4 (per tick)	Creates a persistent tick-based reaction that consumes both elements simultaneously.
Quicken	Dendro / Electro	Dendro / Electro	1.0	Consumes a 1:1 ratio of the trigger's gauge from the aura, creating a new Quicken Aura.
Shatter (via Freeze Aura)	Physical (Blunt)	Freeze Aura	N/A	Consumes the Freeze Aura, shattering the frozen state.

Persistent terrain effects, such as Mudstorm, are implemented through a `TerrainEffectManager` and its associated `TerrainEffectContract`. Unlike auras, which are attached to entities, these effects are part of the platform's authored geometry. The manager reads definitions from content packs and populates a global, persistent state that describes the battlefield's altered properties [13]. For example, the Mudstorm pattern is defined as a specific set of coordinates on the platform's grid. When the `DirectionalLightningPropagator` attempts to route a spell, it queries the `TerrainEffectManager` to check if a potential landing pad falls within an active Mudstorm area. If it does, the propagator's algorithm is modified; for instance, it might increase the probability of branching or introduce a status effect on the hit entity. This tight coupling between the propagation kernel and the terrain manager allows for dynamic, location-dependent spell behavior without altering the core logic of either system. The existence of Mudstorm is now a property of the map itself, not of a specific entity, making it a true environmental mechanic.

Spatial Query Systems and Battle Mode Integration

To enable the complex interactions specified in the EngAIn Combat & Magic Specification, the simulation platform must provide efficient and deterministic methods for querying the game world. This is achieved through a `SpatialQuerySystem`, which is responsible for tasks like finding targets in an Area of Effect (AoE) and determining valid paths for spells. Furthermore, the system must be flexible enough to support different combat rulesets, such as "Offensive Ceiling" and "Realistic Combat." This is

accomplished through a `CombatModeController` that acts as a gatekeeper, mediating between the core game logic and the specific rules of the currently active mode.

The `AoETargetingKernel` is a prime example of a spatial query. Its task is to identify all entities within a defined radius around a central point. A naive approach of checking the distance to every entity in the game world would be computationally expensive. A far more efficient method leverages a sparse spatial hash data structure [36]. This structure partitions the platform's space into a grid of cells. When an entity is created, it is inserted into the hash table based on its position, associating it with the cell(s) its bounding box occupies. To perform an AoE query, the kernel first identifies all grid cells that intersect with the query radius. It then gathers all entities from those cells into a candidate pool. Finally, it performs an accurate distance check only on these candidates to confirm they are truly within the radius. This two-step process drastically reduces the number of expensive distance calculations, making AoE targeting feasible in real-time [36]. The kernel would also respect layer-based collision filtering, excluding entities from specified layers (e.g., projectiles hitting allies) from the final result. The use of an ordered data structure for the final list of valid targets ensures the result is deterministic and consistent across all runs [8].

Pathfinding and obstruction detection for mechanics like Lightning propagation rely on similar spatial query techniques. A `LineOfSightQuery` function, for example, would use a raycasting algorithm to check for obstructions between two points [44]. The choice of algorithm is important; while Bresenham's line algorithm is fast, it can miss grid cells and create asymmetrical vision, potentially leading to exploits [40]. A proper raycasting algorithm that finds every cell a geometric line passes through is more symmetrical and predictable, making it preferable for game mechanics [40]. On a grid-based representation, this can be implemented efficiently using principles from the Digital Differential Analyzer (DDA) algorithm or Bresenham's line rasterization algorithm to iterate through all intersected cells [36, 39]. The query would check each cell in the path against a layer mask to see if it contains an obstacle, returning a boolean result indicating whether a clear path exists [44].

The `CombatModeController` provides a clean abstraction for managing different battle modes. Instead of scattering conditional logic throughout the codebase, this controller centralizes all mode-dependent rules. It holds a reference to the currently active `CombatMode` (e.g., `OFFENSIVE_CEILING`, `REALISTIC_COMBAT`) and exposes methods that the rest of the simulation can call to retrieve the appropriate values. For example, the damage calculation system would not hardcode a damage cap; instead, it would call `combat_mode_controller.get_damage_cap(damage_amount)`. The

controller's internal logic would then select the correct cap table based on the active mode and apply the tiered reduction formula [18]. Similarly, the movement system would call `combat_mode_controller.get_movement_speed_penalty()` to determine if movement is slowed in Realistic Combat mode. This design cleanly separates the core, universal mechanics from the balancing variables, allowing designers to tweak the behavior of the entire system simply by changing the active mode without touching the underlying simulation code. The data contracts for these modes could define tables of thresholds and modifiers, allowing for fine-grained calibration.

Combat Mode	Damage Cap Behavior	Combo Lock	Movement Speed
Offensive Ceiling	Hard or extremely high soft cap on damage.	Combos may be locked to prevent infinite damage loops.	Unaffected.
Realistic Combat	Standard soft damage caps with diminishing returns.	Combos are generally allowed.	May be penalized to simulate encumbrance.

By combining an efficient spatial query system with a well-defined combat mode integration layer, the EngAIn platform can support the complex spatial reasoning required for AoE targeting and pathing, while also providing the flexibility needed for different gameplay experiences and balance philosophies.

Validation Framework and Combat Calibration

A comprehensive validation framework is essential to ensure the correctness and stability of the EngAIn combat system. This framework must provide concrete, executable tests for each major mechanic, moving beyond abstract interfaces to exercise the actual algorithms. These tests, combined with a principled approach to combat calibration, will allow developers and authors to confidently implement and tune the game's core systems.

The validation framework should consist of unit tests written in pseudocode that directly invoke the system's core algorithms with controlled inputs and verify their outputs against explicit success criteria. These tests should be integrated into the project's CI/CD pipeline to catch regressions automatically [15]. The following examples demonstrate how to validate some of the most critical mechanics specified in the user's request.

Test Case: Lightning Triple-Chain Termination This test verifies that a Lightning spell correctly stops propagating after reaching its maximum chain length.

```
// Setup: Load a Lightning spell with a max_chain_length of 3.
//        Place three enemies on a linear path to create a valid chain.
//        Ensure the fourth potential landing spot is obstructed.
spell = loadSpell("LightningStrike", config={max_chain_length: 3})
enemy1 = placeEntityOnPad(pad_chain[0])
enemy2 = placeEntityOnPad(pad_chain[1])
enemy3 = placeEntityOnPad(pad_chain[2])
placeObstructionOnPad(pad_chain[3]) // Block the fourth hit

// Execute the propagation algorithm
hits = DirectionalLightningPropagator.propagate(source_pad, spell)

// Verify Success Criteria:
// 1. The number of hits should be exactly 3, matching the max_chain_lengt
assert(hits.size == 3)

// 2. The third hit should land on the third pad in our chain.
assert(hits[2].pad == pad_chain[2])

// 3. There should be no fourth hit, confirming termination.
assert(not isPathValidAfterHit(hits[2]))
```

Test Case: Mudstorm Trigger Pattern This test confirms that a Lightning spell will preferentially follow a terrain-induced pattern like Mudstorm when available.

```
// Setup: Define a Mudstorm pattern and place it on the platform.
//        Place entities in a straight line <em>and</em> along the Mudstor
//        Ensure no other obstructions exist.
mudstorm_pattern = generateMudstormPattern(platform_id)
terrain_effect_manager.addEffect(TerrainEffectType.MUDSTORM, mudstorm_patt
clearAllEntities()
placeEntitiesAlongMudstormPath() // Entities are on Mudstorm pads
placeStraightLineEntity() // An entity off the Mudstorm path

// Execute the propagation algorithm
hits = DirectionalLightningPropagator.propagate(source_pad, spell)

// Verify Success Criteria:
// 1. The path should have more than 3 hits, demonstrating the extended pa
assert(hits.size > 3)
```

```
// 2. The path must conform to the Mudstorm pattern, not just a straight l
assert(pathFollowsMudstormPattern(hits))

// 3. The hit on the straight-line entity should not be part of the primar
assert(entity_on_straight_line not in hits)
```

Test Case: Melt Reaction Gauge Consumption This test validates the core logic of an elemental reaction, ensuring the correct amount of gauge is consumed.

```
// Setup: Create a target entity and apply a 1.6U Cryo aura to it.
//        Load a Pyro spell with a Melt reaction type and a 1.5U gauge.
target = createTarget()
target.addAura(ElementType.Cryo, gauge_units=1.6)
melt_spell = loadSpell("PyroAttack", gauge_units=1.5, reaction_type=MeltRe

// Execute the elemental reaction
result = ElementalReactionKernel.apply(melt_spell, target)

// Verify Success Criteria:
// 1. A positive amount of damage should be dealt.
assert(result.damage_dealt > 0)

// 2. The gauge consumed should be 1.5U * 2.0 (the Melt coefficient), equa
assert(result.gauge_consumed == approx(1.5 * 2.0))

// 3. The original Cryo aura should still be present but weakened.
assert(target.hasAura(ElementType.Cryo))
assert(target.getAura(ElementType.Cryo).gauge_units < 1.6)
```

In addition to validating mechanics, the platform must support combat calibration. The specification mentions "combat calibration" as a key requirement. This process involves tuning values like damage caps to achieve a desired balance curve. Soft damage caps are typically implemented using a tiered reduction system [18]. The calibration tool would expose a system for defining these tiers. For example, a designer could define a table with thresholds (e.g., 300k, 400k, 500k) and corresponding reduction modifiers (e.g., 0%, 20%, 40%) [18]. The `DamageCalibrator` algorithm would then take this table and calculate the effective damage that gets through each tier. The formula involves summing the damage that passes through each band, which is the width of the band multiplied by the percentage of damage that gets through [18]. This allows for the creation of a curve

with diminishing returns, where damage increases at a decreasing rate past a certain point, preventing runaway power escalation [20]. This calibrated damage value would then be passed to the damage application system. This systematic approach to calibration, grounded in a clear mathematical model, provides a transparent and repeatable way to balance the combat system, directly fulfilling the user's request.

Reference

1. Runtime authority layout - OwendB1/arcane-systems https://github.com/OwendB1/arcane-systems/blob/main/docs/authority-layout.md
2. mas-bandwidth/fixed3d: Box3D physics engine converted ... https://github.com/mas-bandwidth/fixed3d
3. deterministic physics using a fixed-point conversion buffer https://pybullet.org/Bullet/phpBB3/viewtopic.php%3Ft=6831.html
4. Floating Point Determinism https://gafferongames.com/post/floating_point_determinism/
5. Game Engines and Determinism https://www.duality.ai/blog/game-engines-determinism
6. Getting started with SG Physics 2D and deterministic ... https://www.snopekgames.com/tutorial/2021/getting-started-sg-physics-2d-and-deterministic-physics-godot/
7. In some game engines, Fixed Point Arithmetic is used ... https://news.ycombinator.com/item?id=26357209
8. Game Networking Demystified, Part II: Deterministic - Ruoyu Sun https://ruoyusun.com/2019/03/29/game-networking-2.html
9. Server authoritative | Elympics Documentation https://docs.elympics.ai/gameplay/glossary/server-authoritative
10. Server-Authoritative vs Client-Authoritative Game Backend ... https://www.metaplay.io/comparisons/server-authoritative-vs-client-authoritative
11. Making Web Games #5: Authoritative Server - James Long https://archive.jlongster.com/Making-Web-Games--5--Authoritative-Server
12. Server-authoritative setup https://docs.coherence.io/manual/networking-state-changes/authority/server-authoritative-setup

13. engine-architecture.md - Erotemic/ambition https://github.com/Erotemic/ambition/blob/main/docs/architecture/engine-architecture.md
14. SpellData · simulationcraft/simc Wiki https://github.com/simulationcraft/simc/wiki/SpellData
15. Data Contracts - Key Components, Use Cases, & Examples https://www.getcollate.io/learning-center/data-contracts
16. Tech Breakdown: AI with Finite State Machines - Little Polygon ... https://blog.littlepolygon.com/posts/fsm/
17. (PDF) Finite State Machine in Game Development https://www.researchgate.net/publication/355518086_Finite_State_Machine_in_Game_Development
18. Damage Cap https://gbf.wiki/Damage_Cap
19. Many games use a formula to calculate damage mitigation ... https://www.instagram.com/reel/Ddj3Q8EgSku/
20. Damage cap, the only solution to balance? https://us.forums.blizzard.com/en/d4/t/damage-cap-the-only-solution-to-balance/224971
21. Cross-Platform Floating-Point Determinism Out of the Box https://www.youtube.com/watch?v=7MatbTHGG6Q
22. mrdav30/FixedMathSharp: A deterministic fixed-point math ... https://github.com/mrdav30/FixedMathSharp
23. Fixed-Point Math - Metaplay Docs https://docs.metaplay.io/game-logic/utilities/fixed-point-math
24. Determinism, prediction and rollback https://docs.coherence.io/manual/advanced-topics/competitive-games/determinism-prediction-rollback
25. Finite State Machines in Game Development https://jessewarden.com/2012/07/finite-state-machines-in-game-development.html
26. Turn-Based Combat State Machine Design - grasp.study https://grasp.study/modules/2f8c1f0a-3ff4-43a7-aa66-823994f92082/lessons/6053c853-337d-4985-a020-ead1a14332d4
27. enforce deterministic same-seed simulation across replay ... https://github.com/a1flecke/conquestoria/issues/1004
28. [2607.16200] Deterministic Replay for AI Agent Systems https://arxiv.org/abs/2607.16200
29. Understanding Framedata: Combos, Traps, and Turns https://critpoints.net/2016/11/29/understanding-framedata-combos-traps-and-turns/
30. Tutorial: How to Test Frame Data https://8wayrun.com/threads/tutorial-how-to-test-frame-data.12706/

31. Fighting Game Motion Input Programming https://tmptesting.godotforums.randommomentania.com/d/25748-fighting-game-motion-input-programming
32. How to Code Fighting Game Motion Inputs https://critpoints.net/2025/02/05/how-to-code-fighting-game-motion-inputs/
33. How to design a replay system https://gamedev.stackexchange.com/questions/6080/how-to-design-a-replay-system
34. Replay Boundaries in a Deterministic Game Engine https://www.reddit.com/r/gameenginedevs/comments/1tobspm/not_every_byte_gets_a_vote_replay_boundaries_in_a/
35. Deterministic Replay: How to Debug AI Agents That Never ... https://tianpan.co/blog/2026/04/12/deterministic-replay-debugging-non-deterministic-ai-agents
36. Spatial Hashing https://gamedev.net/tutorials/programming/general-and-gameplay-programming/spatial-hashing-r2697
37. Detecting obstacles along a line - A* Pathfinding Project https://arongranberg.com/astar/documentation/beta/linecasting.html
38. More efficient way to implement Line of sight on a 2d grid ... https://gamedev.stackexchange.com/questions/47576/more-efficient-way-to-implement-line-of-sight-on-a-2d-grid-with-ray-casting
39. Raycasting https://lodev.org/cgtutor/raycasting.html
40. Bresenham's or Raycasting for Vision - Sam Driver https://samdriver.xyz/article/line-of-sight-on-grid
41. Bresenham algorithm https://deepnight.net/tutorial/bresenham-magic-raycasting-line-of-sight-pathfinding/
42. Elemental Gauge Theory - Genshin Impact Wiki - Fandom https://genshin-impact.fandom.com/wiki/Elemental_Gauge_Theory
43. Elemental Gauge Theory https://library.keqingmains.com/combat-mechanics/elemental-effects/elemental-gauge-theory
44. Collision queries | Unity Physics | 1.0.16 https://docs.unity3d.com/Packages/com.unity.physics@1.0/manual/collision-queries.html
45. Deterministic simulation, record and replay | Balaur https://balaurengine.org/docs/manual/determinism/
46. Deterministic Simulation Testing for Our Entire SaaS https://www.warpstream.com/blog/deterministic-simulation-testing-for-our-entire-saas
47. A Deterministic Simulation Testing (DST) primer https://www.amplifypartners.com/blog-posts/a-dst-primer-for-unit-test-maxxers

48. Will Wilson on Deterministic Simulation Testing https://se-radio.net/2025/09/se-radio-685-will-wilson-on-deterministic-simulation-testing/
49. A Deterministic Simulation Testing (DST) Journey https://qconlondon.com/presentation/mar2026/deterministic-simulation-testing-dst-journey-wasm-go-state-machines-rust
50. 601.229 (F21): Assignment 1: Fixed-point arithmetic https://jhucsf.github.io/fall2021/assign/assign01.html
51. [Blog Link] Fixed point arithmetic in C#/Unity for RTS games https://discussions.unity.com/t/blog-link-fixed-point-arithmetic-in-c-unity-for-rts-games/652108